

Machine Learning Algorithms and Applications

Lecture 7

(Block 3)

Plan for Today



Unsupervised Learning



mean-shift



Clustering



Hierarchical Clustering



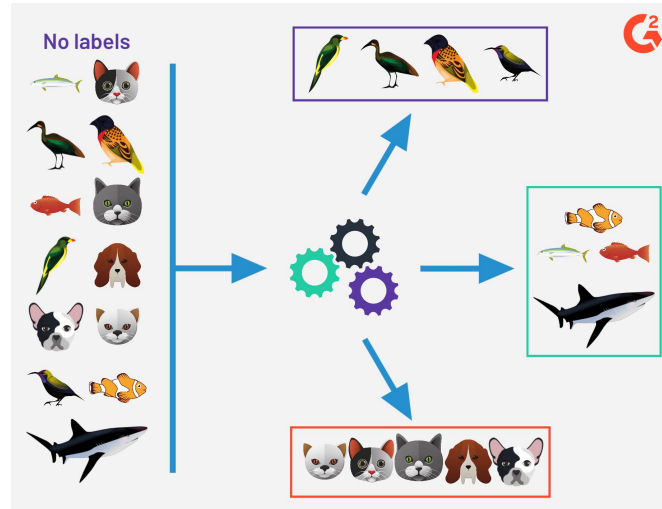
Lab



Unsupervised Learning

Unsupervised Learning

Refers to the use of machine learning algorithms to **identify patterns** in datasets containing observations that are **neither classified nor labeled**.



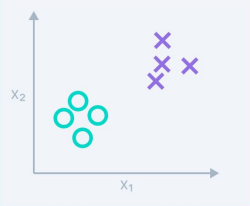
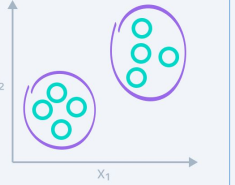
Supervised vs Unsupervised Learning

Supervised:

Each observation has a ground truth. Models learn pattern in order to predict this target variable.

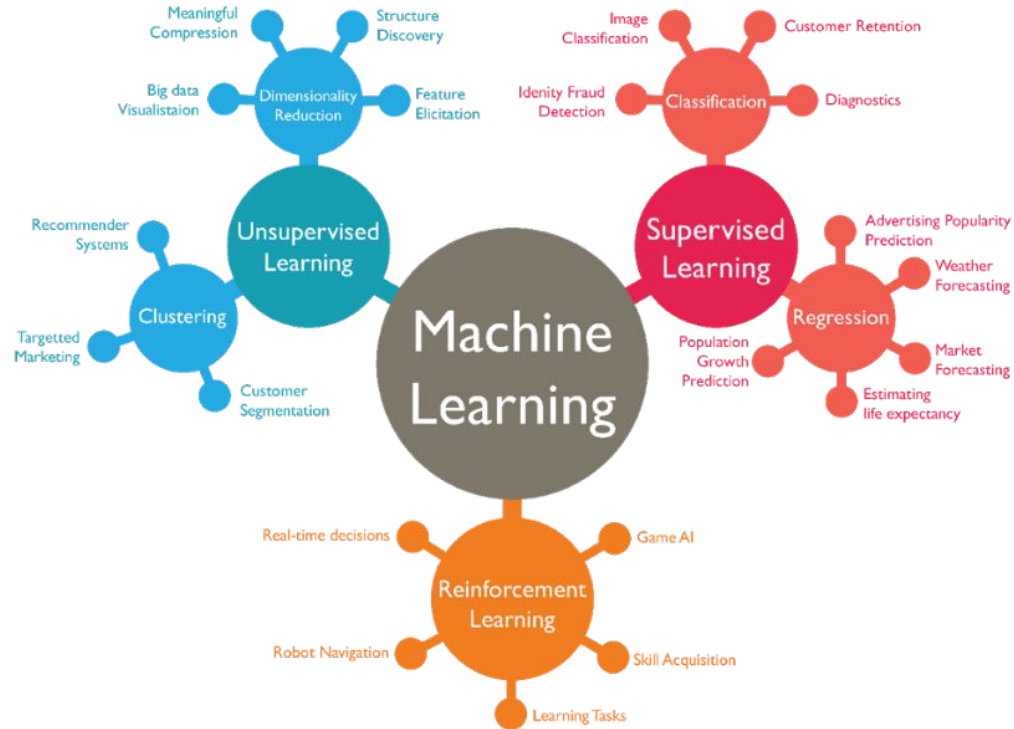
Unsupervised:

There is no ground truth in the dataset. Models identify natural patterns on its own.

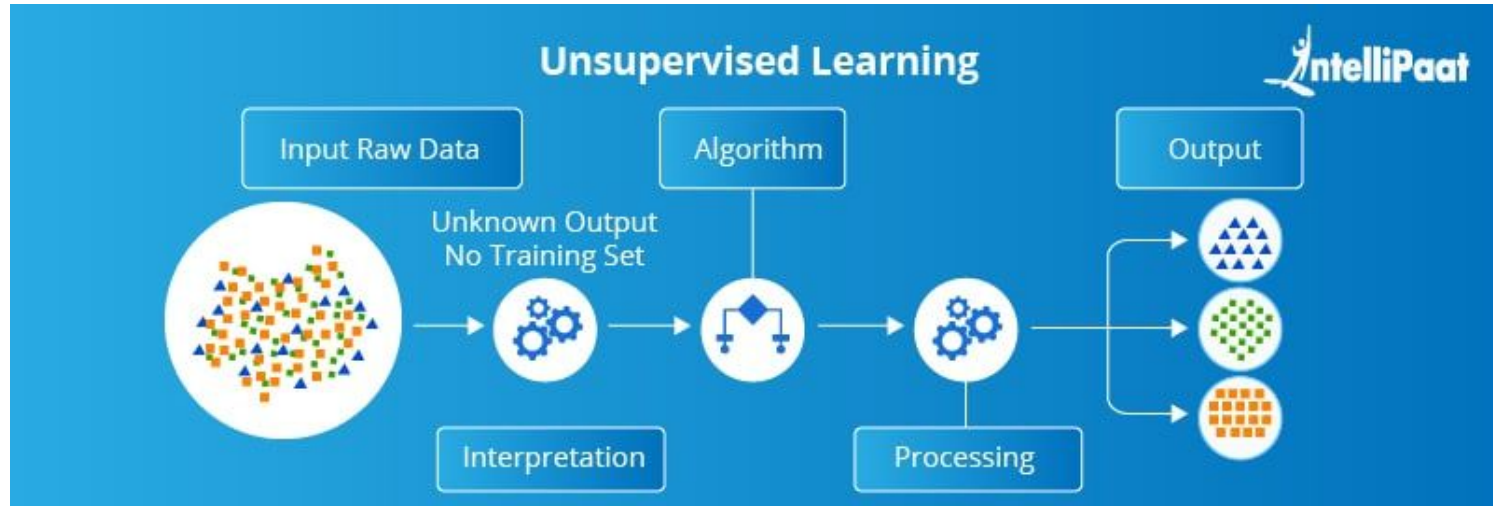
Supervised learning	Unsupervised learning
Input data is labeled	Input data is unlabeled
Has a feedback mechanism	Has no feedback mechanism
Data is classified based on the training dataset	Assigns properties of given data to classify it
Divided into Regression & Classification	Divided into Clustering & Association
Used for prediction	Used for analysis
Algorithms include: decision trees, logistic regressions, support vector machine	Algorithms include: k-means clustering, hierarchical clustering, apriori algorithm
A known number of classes	A unknown number of classes
	

V7 Labs

Use Cases



Training Process



No need to split into training, validation and test sets.

No need to assess model performance

Can be used as analytic tool

If patterns learned have business meaning/value can be used on unseen data

Algorithms

	ALGORITHM	DESCRIPTION & APPLICATION	ADVANTAGES	DISADVANTAGES
Clustering	K-Means	Most common clustering approach which assumes that the closer data points are to each other, the more similar they are. It determines K clusters based on Euclidean distances.	1. Scales to large datasets 2. Interpretable & explainable results 3. Can generate tight clusters	1. Requires defining the expected number of clusters in advance. 2. Not suitable to identify clusters with non-convex shapes.
	DBSCAN	Density-Based Spatial Clustering of Applications with Noise can handle non-linear cluster structures, purely based on density. It can differentiate and separate regions with varying degrees of density, thereby creating clusters.	1. No assumption on the expected number of clusters. 2. Can handle noisy data and outliers 3. No assumptions on the shapes and sizes of the clusters 4. Can identify clusters with different densities	1. Requires optimization of two parameters 2. Can struggle in case of very high dimensional data
	HDBSCAN	Family of the density-based algorithms and has roughly two steps: finding the core distance of each point, and expands clusters from them. It extends DBSCAN by converting it into a hierarchical clustering algorithm.	1. No assumption on the expected number of clusters 2. Can handle noisy data and outliers. 3. No assumptions on the shapes and sizes of the clusters. 4. Can identify clusters with different densities	1. Mapping of unseen objects in HDBSCAN is not straightforward. 2. Can be computationally expensive
	Agglomerative Hierarchical Clustering	Uses hierarchical clustering to determine the distance between samples based on the metric, and pairs are merged into clusters using the linkage type.	1. There is no need to specify the number of clusters. 2. With the right linkage, it can be used for the detection of outliers. 3. Interpretable results using dendrograms.	1. Specifying metric and linkages types requires good understanding of the statistical properties of the data 2. Not straightforward to optimize 3. Can be computationally expensive for large datasets
	OPTICS	Family of the density-based algorithms where it finds core sample of high density and expands clusters from them. It operates with a core distance (ϵ) and reachability distance.	1. No assumption on the expected number of clusters. 2. Can handle noisy data and outliers. 3. No assumptions on the shapes and sizes of the clusters. 4. Can identify clusters with different densities. 5. Not required to define a fixed radius as in DBSCAN.	1. It only produces a cluster ordering. 2. Does not work well in case of very high dimensional data. 3. Slower than DBSCAN.
	Gaussian Mixture Models	Gaussian Mixture Models (GMM) leverages probabilistic models to detect clusters using a mixture of normal (gaussian) distributions.	1. Provides uncertainty measures for each observation 2. Can identify overlapping clusters	1. Requires defining the expected number of clusters or mixture components in advance 2. The covariance type needs to be defined for the mixture of components
Dimensionality Reduction	PCA	Principal Component Analysis (PCA) is a feature extraction approach that uses a linear function to reduce dimensionality in datasets by minimizing information loss.	1. Explainable & interpretable results. 2. New unseen datapoints can be mapped into the existing PCA space. 3. Can be used as a dimensionality reduction technique as a preliminary step to other machine learning tasks 4. Helps reduce overfitting 5. Helps remove correlated features	1. Sensitive to outliers 2. Requires data standardization
	t-SNE	t-distributed Stochastic Neighbor Embedding is a non-linear dimensionality reduction method that converts similarities between data points to joint probabilities using the Student t-distribution in the low-dimensional space	1. Helps preserve the relationships seen in high dimensionality 2. Easy to visualise the structure of high dimensional data in 2 or 3 dimensions 3. Very effective for visualizing clusters or groups of data points and their relative proximities	1. The cost function is not convex; different initializations can get different results. 2. Computationally intensive for large datasets. 3. Default parameters do not always achieve the best results
	UMAP	Uniform Manifold Approximation and Projection (UMAP) constructs a high-dimensional graph representation of the data then optimizes a low-dimensional graph to be as structurally similar as possible.	1. It can be used as a general-purpose dimension reduction technique as a preliminary step to other machine learning tasks. 2. Can be very effective for visualizing clusters or groups of data points and their relative proximities. 3. Able to handle high dimensional sparse datasets	1. Default parameters do not always achieve the best results
	ICA	Independent Component Analysis (ICA) is a linear dimensionality reduction method that aims to separate a multivariate signal into additive subcomponents under the assumption that independent components are non-gaussian. Where PCA "compresses" the data, ICA "separates" the information.	1. Can separate multivariate signals into its subcomponents. 2. Clear aim of the method; only applicable if there are multiple independent generators of information to uncover. 3. Can extract hidden factors in the data by transforming a set of variables to a new set that is maximally independent.	1. Without any prior knowledge, determination of the number of independent components or sources can be difficult. 2. PCA is often required as a pre-processing step.
	PaCMAP	Pairwise Controlled Manifold Approximation (PaCMAP) is a dimensionality reduction method that optimizes low-dimensional embeddings using three kinds of point pairs: neighbor pairs, mid-near pair, and further pairs.	1. It can preserve both local and global structure of the data in original space. 2. Performance is relatively robust within reasonable parameter choices.	1. Parameters have been tuned on smaller datasets and it is yet unknown how it behaves and extends to very high-dimensional datasets



Clustering

What is a Clustering Task



Unsupervised learning

Doesn't requires a target variable



Objective

Identify relationship between observations (i.e. grouping) based on their features



Applications

(Retail) Customer segmentation
(Finance) Product usage
(Security) Anomaly detection

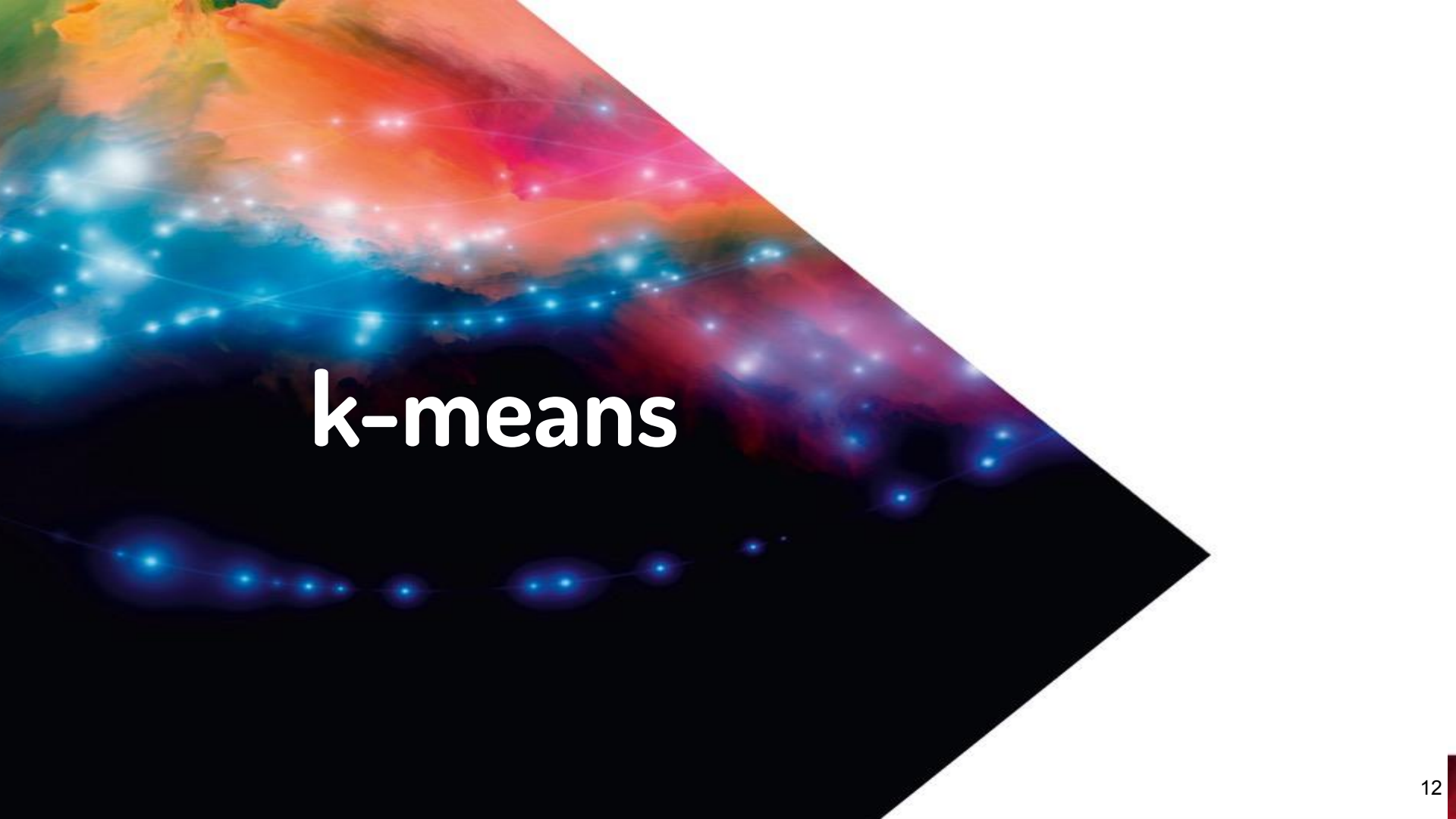


Clustering Example

Segmenting Usage Behavior

Features

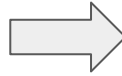
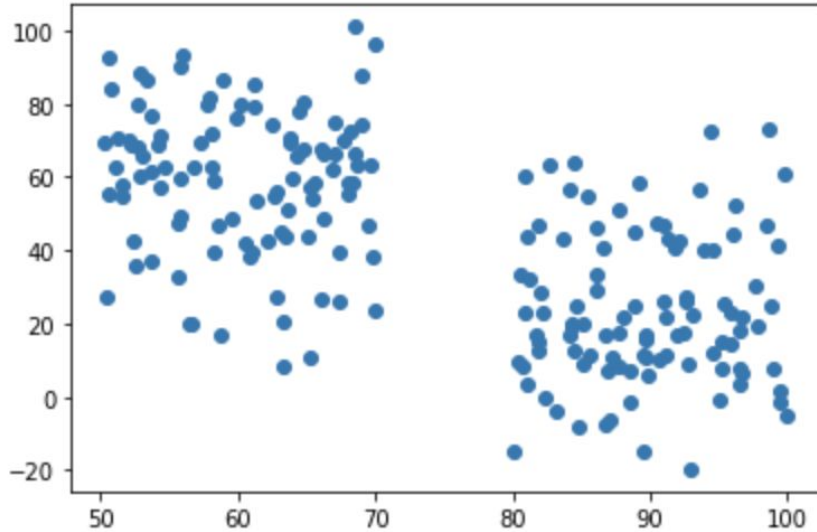
Months since first subscription	Monthly average spent	Average number of phone calls made last month	Average number of phone calls made last quarter	Additional options
13	\$70	56	63	Yes
2	\$35	35	34	Yes
6	\$40	46	50	Yes
16	\$110	53	75	No



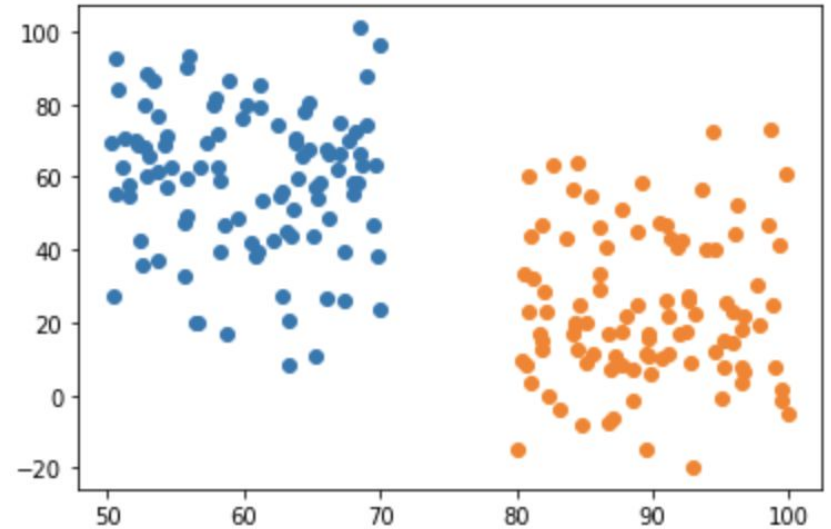
k-means

Grouping of data

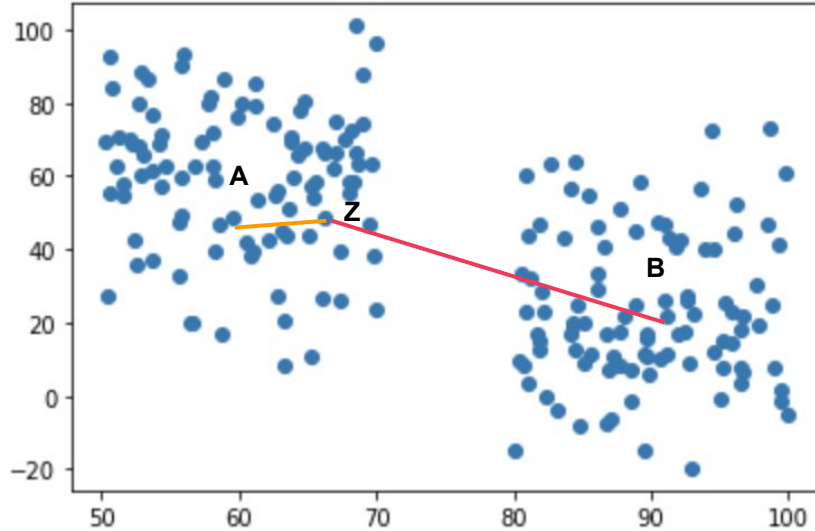
Raw data



Clusters



Data Points Proximity



Distance between A and Z is shorter than Distance between B and Z
(ie Z is closer to A than B)

Z is more likely to belong to the same group of data points as A

If we repeat this process for all observations we can group them based on their proximity

Euclidean Distance

Formula:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

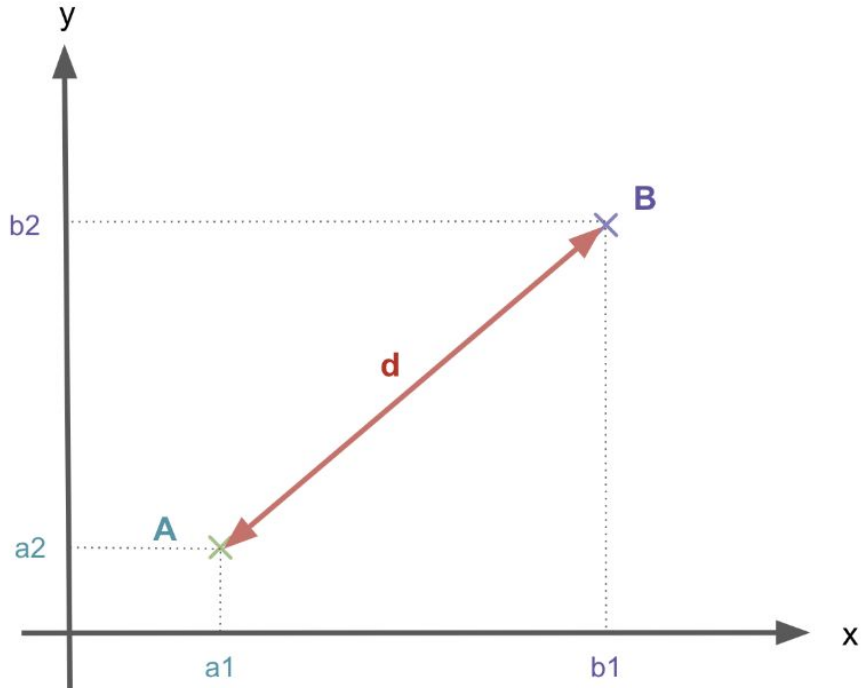
$$A = (1674, 208)$$

$$B = (1292, 530)$$

$$d(A, B) = \sqrt{(A_1 - B_1)^2 + (A_2 - B_2)^2}$$

$$= \sqrt{(1674 - 1292)^2 + (208 - 530)^2}$$

$$= \sqrt{(382)^2 + (-322)^2} = \sqrt{249608} = 499.61$$



Manhattan Distance

Formula:

$$d(A, B) = \sum_{i=1}^n |A_i - B_i|$$

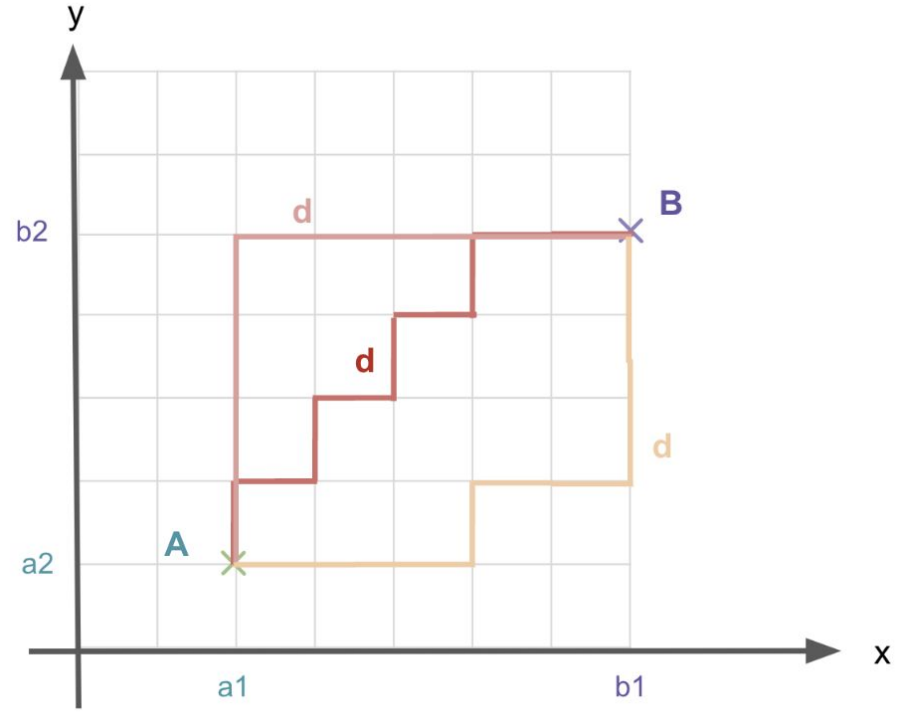
$$A = (1674, 208)$$

$$B = (1292, 530)$$

$$d(A, B) = |A_1 - B_1| + |A_2 - B_2|$$

$$= |1674 - 1292| + |208 - 530|$$

$$= 382 + 322 = 704$$

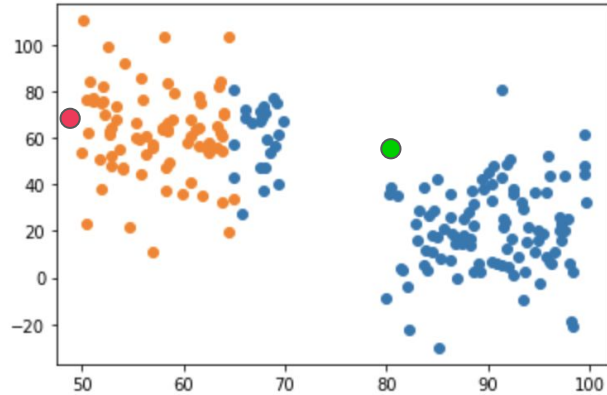


Centroids

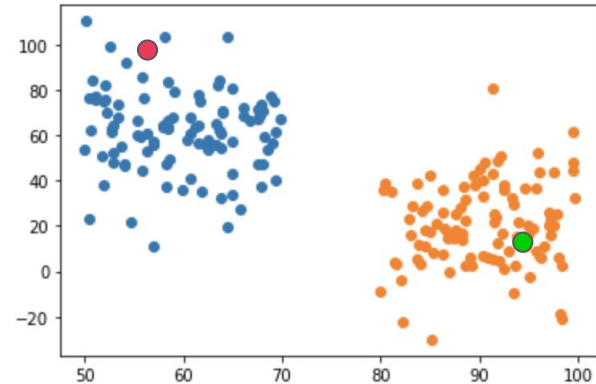
Definition:

Center of a cluster. The distance between each observation and a centroid will be measured in order to assign it the closest cluster

How do we choose these centroids?



Randomly



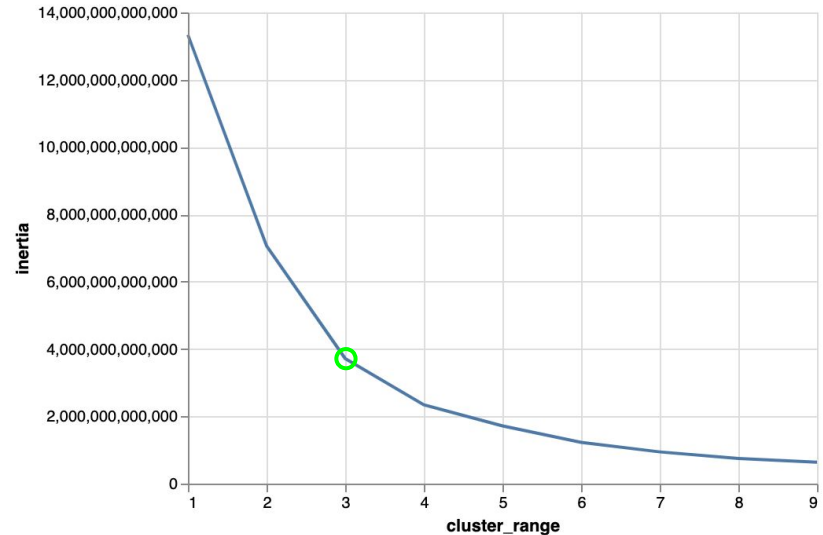
kmeans++

How many clusters

Elbow Method:

For a range of number of clusters, calculate the inertia (measure how far data points are from centroids) and plot it on a graph

Find the inflexion point (45 degrees), where the slope of inertia starts to decrease. This will be the optimal number of clusters.



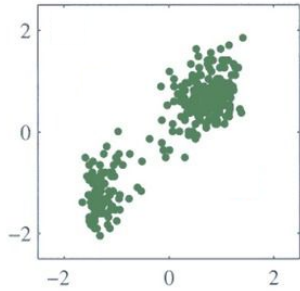
Kmeans Algorithm

Pseudo-algorithm:

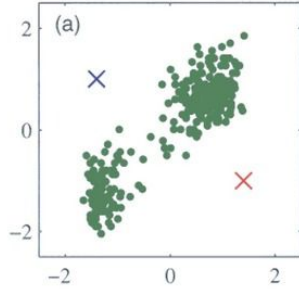
1. Set the value K for the number of clusters (n_clusters).
2. Choose K data points from the dataset as the initial centroids of the clusters (init).
3. For each observation:
 - a. Calculate its distance to the centroids
 - b. Assign this observation to the group with the closest centroid
4. Calculate the center point of each cluster. This will become the new centroid
5. Repeat step 3 and 4 until either the centroids don't move anymore or the maximum iteration threshold is met (max_iter)

Repeat this process several times (n_init) and choose the best clustering (minimum inertia)

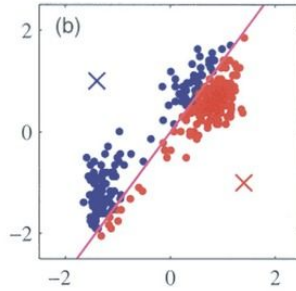
Kmeans Illustration 1



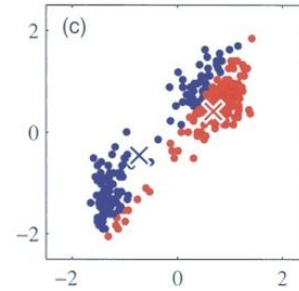
Load data



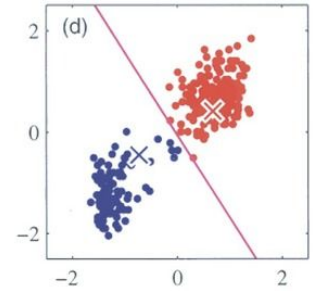
Assign k centroids



Calculate distance to centroids and assign to closest one

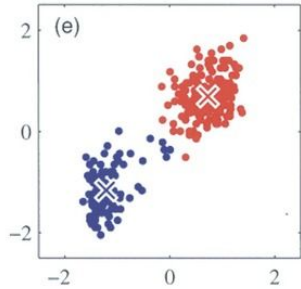


Update centroids based assigned observations

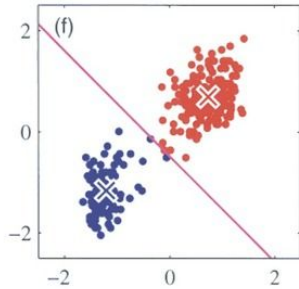


Calculate distance to new centroids and assign to closest one

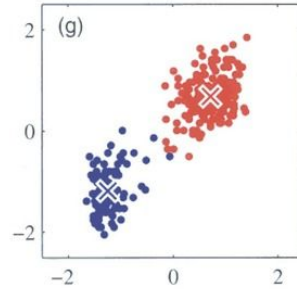
Kmeans Illustration 2



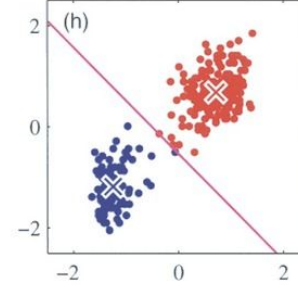
Update centroids based
assigned observations



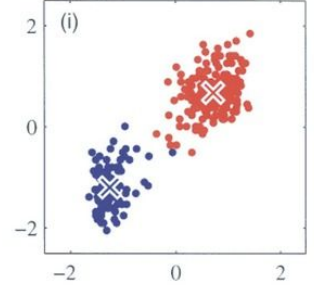
Calculate distance to new
centroids and assign to
closest one



Update centroids based
assigned observations



Calculate distance to new
centroids and assign to
closest one



Save final centroids found



Scikit-learn Kmeans

Generic API for sklearn models:



Import class

```
from sklearn.cluster import KMeans
```



Instantiate a model with hyperparameter

```
km = KMeans(random_state=1, n_clusters=3, init='k-means++', n_init=50)
```



Fit a model

```
km.fit(X)
```



Predict with a model

```
km.predict(new_X)
```



Calculate inertia

```
km.inertia_
```


Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

Parameters: **n_clusters : int, default=8**

The number of clusters to form as well as the number of centroids to generate.

init : {'k-means++', 'random', ndarray, callable}, default='k-means++'

Method for initialization:

'k-means++': selects initial cluster centers for k-mean clustering in a smart way to speed up convergence.

See section Notes in k_init for more details.

'random': choose `n_clusters` observations (rows) at random from data for the initial centroids.

If an ndarray is passed, it should be of shape (n_clusters, n_features) and gives the initial centers.

If a callable is passed, it should take arguments X, n_clusters and a random state and return an initialization.

n_init : int, default=10

Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of n_init consecutive runs in terms of inertia.

max_iter : int, default=300

Maximum number of iterations of the k-means algorithm for a single run.

tol : float, default=1e-4

Relative tolerance with regards to Frobenius norm of the difference in the cluster centers of two consecutive iterations to declare convergence. It's not advised to set `tol=0` since convergence might never be declared due to rounding errors. Use a very small number instead.



mean-shift

Mean Shift Pseudo Algorithm

For **point** in **observations**:

While **point** is not stable:

 Perform shift(**point**, **observations**)

Assign cluster

Perform shift(**point**, **observations**):

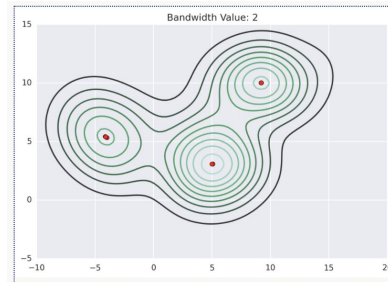
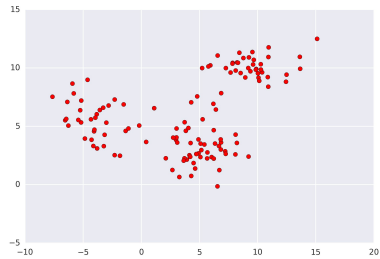
For **obs** in **observations**:

 Calculate distance between **point** and **obs**

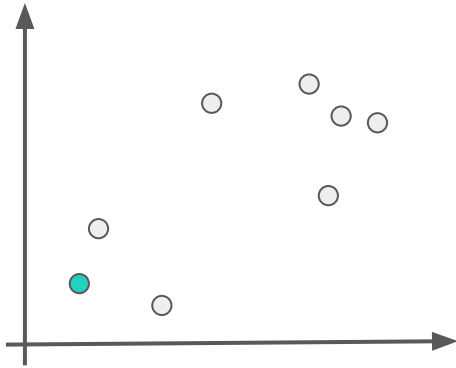
 Keep obs with distance below specified radius

 Calculate mean of filtered **obs**

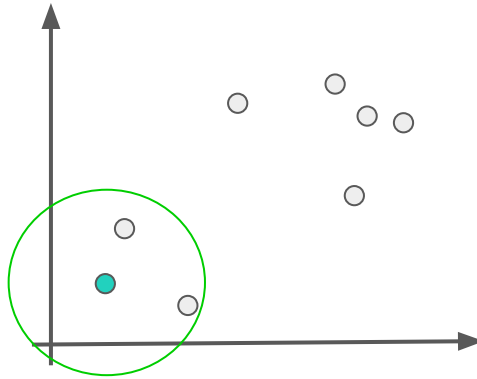
Return mean



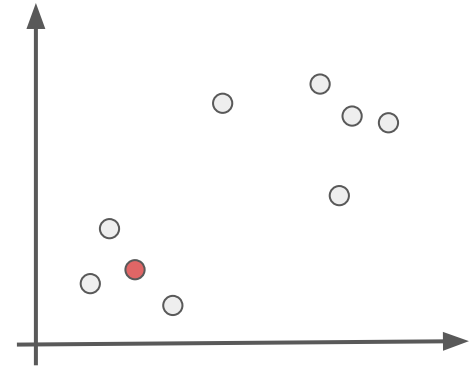
Mean Shift Illustration 1



Select a point

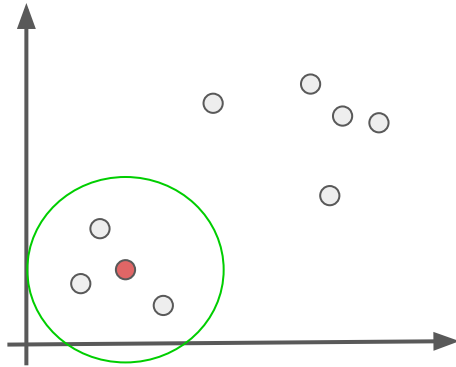


Calculate distance and
select points within radius

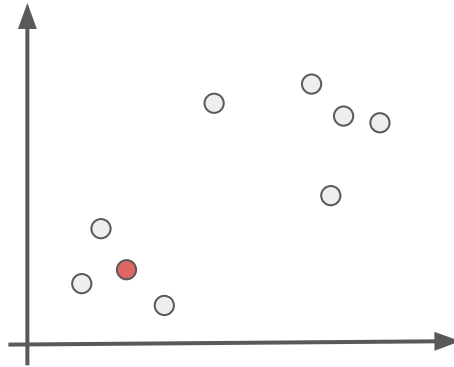


Calculate mean of selected
points

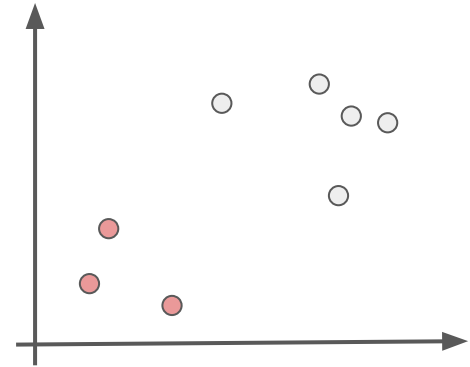
Mean Shift Illustration 2



Calculate distance and
select points within radius

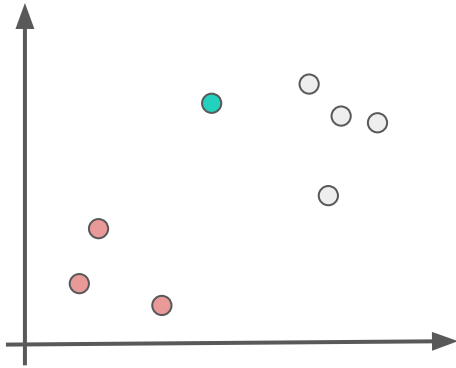


Calculate mean. Mean is
stable (hasn't moved)

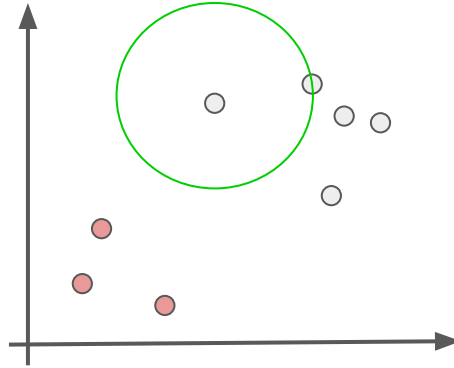


Assign cluster

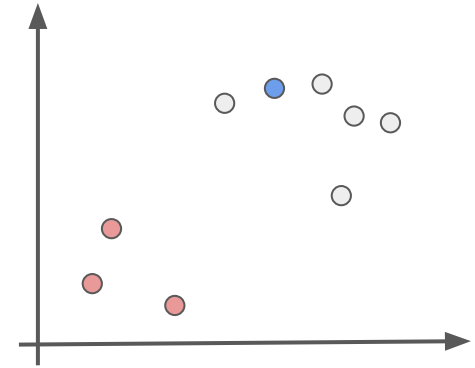
Mean Shift Illustration 3



Select a point



Calculate distance and
select points within radius



Calculate mean of selected
points

Keep iterating until convergence (ie all means of clusters are stable)

Scikit-learn mean-shift

Generic API for sklearn models:



Import class

```
from sklearn.cluster import MeanShift
```



Instantiate a model with
hyperparameter

```
ms = MeanShift()
```



Fit a model

```
ms.fit(X)
```



Predict with a model

```
ms.predict(new_X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.MeanShift.html?highlight=means%20shift#sklearn.cluster.MeanShift>

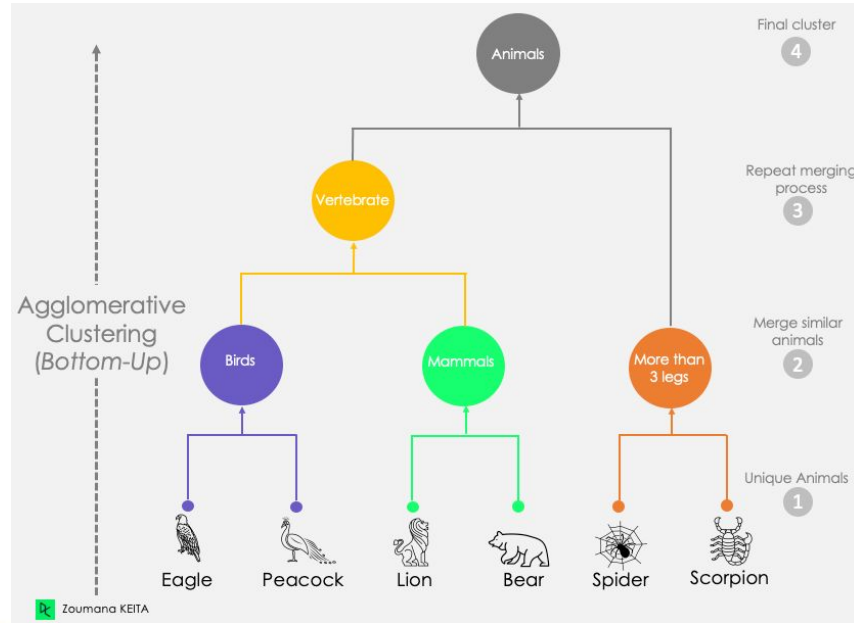
Parameters:	bandwidth : <i>float, default=None</i> Bandwidth used in the RBF kernel. If not given, the bandwidth is estimated using <code>sklearn.cluster.estimate_bandwidth</code> ; see the documentation for that function for hints on scalability (see also the Notes, below).
	seeds : <i>array-like of shape (n_samples, n_features), default=None</i> Seeds used to initialize kernels. If not set, the seeds are calculated by <code>clustering.get_bin_seeds</code> with bandwidth as the grid size and default values for other parameters.
	bin_seeding : <i>bool, default=False</i> If true, initial kernel locations are not locations of all points, but rather the location of the discretized version of points, where points are binned onto a grid whose coarseness corresponds to the bandwidth. Setting this option to True will speed up the algorithm because fewer seeds will be initialized. The default value is False. Ignored if seeds argument is not None.
	min_bin_freq : <i>int, default=1</i> To speed up the algorithm, accept only those bins with at least min_bin_freq points as seeds.
	cluster_all : <i>bool, default=True</i> If true, then all points are clustered, even those orphans that are not within any kernel. Orphans are assigned to the nearest kernel. If false, then orphans are given cluster label -1.



Hierarchical Clustering

Hierarchical Clustering

Hierarchical cluster analysis or **HCA** is an unsupervised clustering algorithm which involves creating clusters that have predominant ordering from top to bottom.

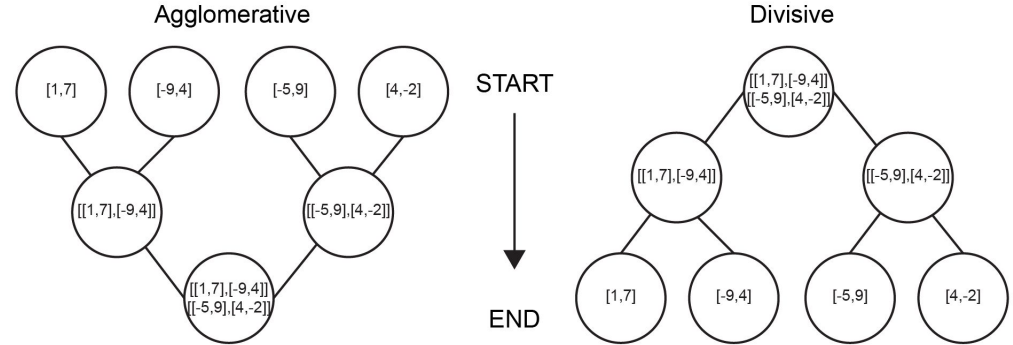


Clustering Techniques

The algorithm groups similar objects into groups called **clusters**.

This clustering technique is divided into two types:

1. Agglomerative Hierarchical Clustering
2. Divisive Hierarchical Clustering



Agglomerative HCA

Most common type of hierarchical clustering used to group objects in clusters based on their similarity starting from the bottom moving to the top: each observation starts in its own cluster, and are merged with others as we move up the hierarchy.

Pseudo-Algorithm:

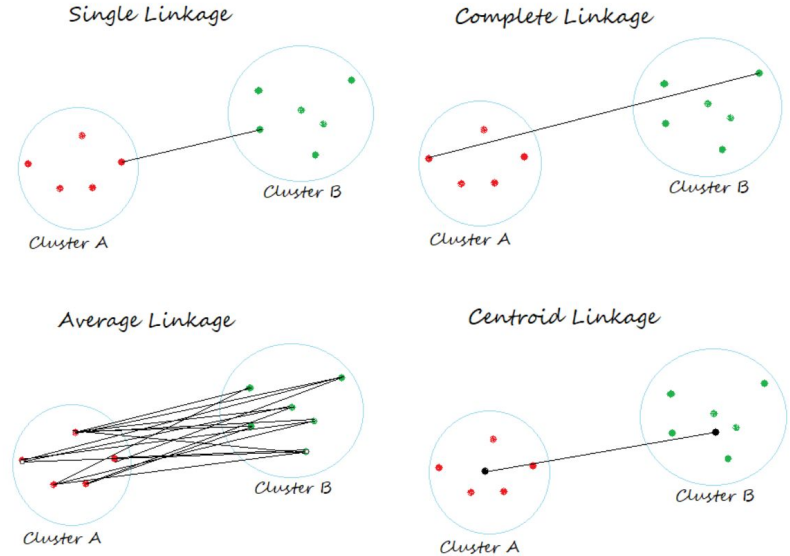
1. Make each data point a single-point cluster → forms N clusters
2. Take the two closest data points and merge them as one cluster → forms $N-1$ clusters
3. Take the two closest clusters and merge them as one cluster → Forms $N-2$ clusters.
4. Repeat step 3 until you are left with only one cluster.



Linkage Methods

Different ways to measure the distance between clusters (called Linkage Methods) such as:

- **Single-linkage:** Measure the distance between all observations from two clusters and keep the shortest one.
- **Complete-linkage:** Measure the distance between all observations from two clusters and keep the longest one.
- **Average-linkage:** Measure the distance between all observations from two clusters and then calculate their average.
- **Centroid-linkage:** Calculate the centroid of 2 clusters and then measure the distance between them.



Scikit-learn AgglomerativeClustering

Generic API for sklearn models:



Import class

```
from sklearn.cluster import AgglomerativeClustering
```



Instantiate a model with hyperparameter

```
model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
```



Fit a model

```
model.fit(X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.AgglomerativeClustering.html>

Parameters: **n_clusters** : *int or None, default=2*

The number of clusters to find. It must be `None` if `distance_threshold` is not `None`.

affinity : *str or callable, default='euclidean'*

The metric to use when calculating distance between instances in a feature array. If metric is a string or callable, it must be one of the options allowed by `sklearn.metrics.pairwise_distances` for its metric parameter. If linkage is "ward", only "euclidean" is accepted. If "precomputed", a distance matrix (instead of a similarity matrix) is needed as input for the fit method.

Deprecated since version 1.2: affinity was deprecated in version 1.2 and will be renamed to `metric` in 1.4.

metric : *str or callable, default=None*

Metric used to compute the linkage. Can be "euclidean", "l1", "l2", "manhattan", "cosine", or "precomputed". If set to `None` then "euclidean" is used. If linkage is "ward", only "euclidean" is accepted. If "precomputed", a distance matrix is needed as input for the fit method.

New in version 1.2.

compute_full_tree : *'auto' or bool, default='auto'*

Stop early the construction of the tree at `n_clusters`. This is useful to decrease computation time if the number of clusters is not small compared to the number of samples. This option is useful only when specifying a connectivity matrix. Note also that when varying the number of clusters and using caching, it may be advantageous to compute the full tree. It must be `True` if `distance_threshold` is not `None`. By default `compute_full_tree` is "auto", which is equivalent to `True` when `distance_threshold` is not `None` or that `n_clusters` is inferior to the maximum between 100 or $0.02 * n_samples$. Otherwise, "auto" is equivalent to `False`.

linkage : *{'ward', 'complete', 'average', 'single'}, default='ward'*

Which linkage criterion to use. The linkage criterion determines which distance to use between sets of observation. The algorithm will merge the pairs of cluster that minimize this criterion.

- 'ward' minimizes the variance of the clusters being merged.
- 'average' uses the average of the distances of each observation of the two sets.
- 'complete' or 'maximum' linkage uses the maximum distances between all observations of the two sets.
- 'single' uses the minimum of the distances between all observations of the two sets.

New in version 0.20: Added the 'single' option

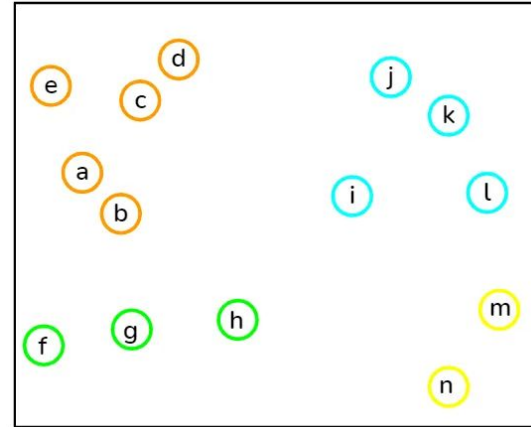
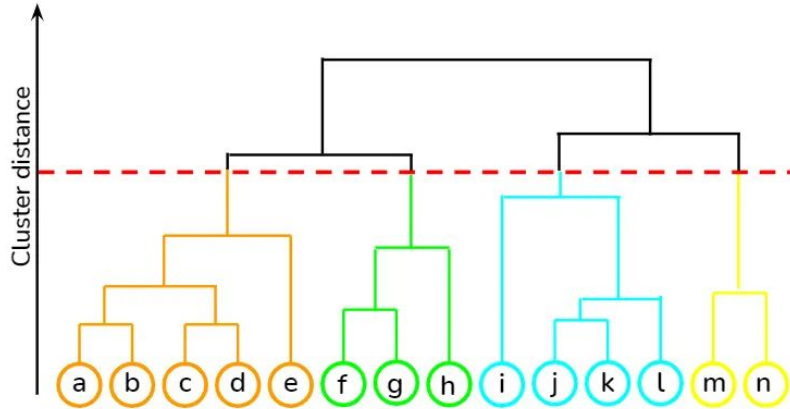
distance_threshold : *float, default=None*

The linkage distance threshold at or above which clusters will not be merged. If not `None`, `n_clusters` must be `None` and `compute_full_tree` must be `True`.

New in version 0.21.

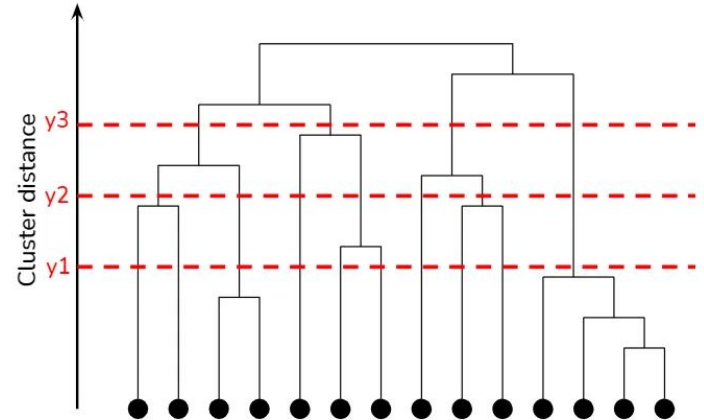
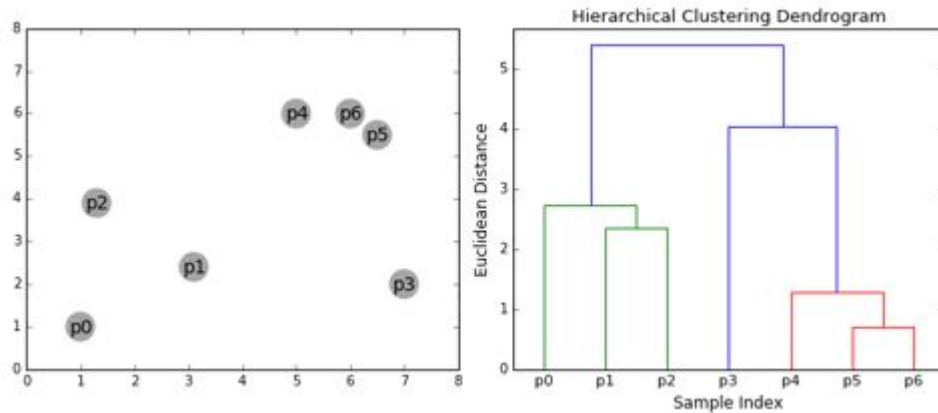
Dendrogram

Tree-like structure that explains the relationship between all the data points from the hierarchical clusters. Looking at a dendrogram, you can understand how the hierarchical clusters are formed.



Dendrogram Levels

Navigating through the levels of a dendrogram (corresponding to a single iteration of the algorithm), you can see what clusters were formed at this stage.

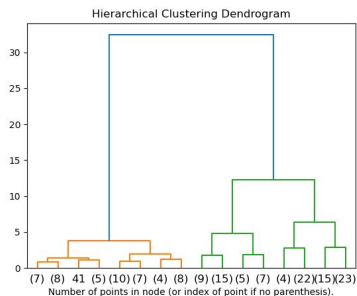


Dendrogram in Code

There is no built-in class in Sklearn to display a dendrogram.

But they provide a tutorial to create one using numpy and matplotlib packages:

https://scikit-learn.org/stable/auto_examples/cluster/plot_agglomerative_dendrogram.html



```
import numpy as np
from matplotlib import pyplot as plt
from scipy.cluster.hierarchy import dendrogram

def plot_dendrogram(model, **kwargs):
    # Create Linkage matrix and then plot the dendrogram

    # create the counts of samples under each node
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1 # Leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count

    linkage_matrix = np.column_stack(
        [model.children_, model.distances_, counts]
    ).astype(float)

    # Plot the corresponding dendrogram
    dendrogram(linkage_matrix, **kwargs)
```



Assignment

Assignment 3

Objective:

A Telecom company has hired you as their first data scientists. They have been collecting transactional data from their customers for the last quarter. They are not well versed in Data Science nor Machine Learning.

Your team has been tasked to explore existing datasets and propose machine learning use cases that will bring value directly to the business or the end customers.

Each member of your team has to define a business problem from the shared datasets, train relevant Machine Learning algorithms and showcase results achieved. You have to provide at least one use case for each type of the following data mining problem:

- Regression analysis (student 1)
- Classification analysis (student 2)
- Clustering analysis (student 3)
- Anomaly detection (student 4)

Student	EDA Notebooks	Model Training Notebooks
Student 1	36106-25AU-AT3-group<id>-<student_id1>-eda-customer_details.ipynb	36106-25AU-AT3-group<id>-<student_id1>-regression.ipynb
Student 2	36106-25AU-AT3-group<id>-<student_id2>-eda-customer_address.ipynb 36106-AT3-group<id>-<student_id2>-eda-customer_satisfaction.ipynb	36106-25AU-AT3-group<id>-<student_id2>-classification.ipynb
Student 3	36106-25AU-AT3-group<id>-<student_id3>-eda-customer_subscriptions.ipynb	36106-25AU-AT3-group<id>-<student_id3>-clustering.ipynb
Student 4	36106-25AU-AT3-group<id>-<student_id4>-eda-customer_usage.ipynb	36106-25AU-AT3-group<id>-<student_id4>-anomaly.ipynb

Assignment 3

Here are some examples of potential use cases:

- Helping Finance Team to adjust budget by predicting next month revenue (regression)
- Helping Sales Support team to target customers most likely to churn in the next 3 months (classification)
- Helping Marketing Team to send customised marketing emails to groups of customers presenting similar usage behaviors (clustering)
- Helping Customer Support team to reach out to customers with abnormal spending patterns (anomaly detection)
-

Submission:

- Assigned Jupyter Notebooks
- A final report reflecting on the project, each business uses cases chosen, approaches taken, experiments performed and their outcome and the benefits and risks for the customers or business units. The report should not exceed 10,000 words. Use the steps defined by the CRISP-DM methodology for the sections of your report.

Assignment 3

SparkPlus

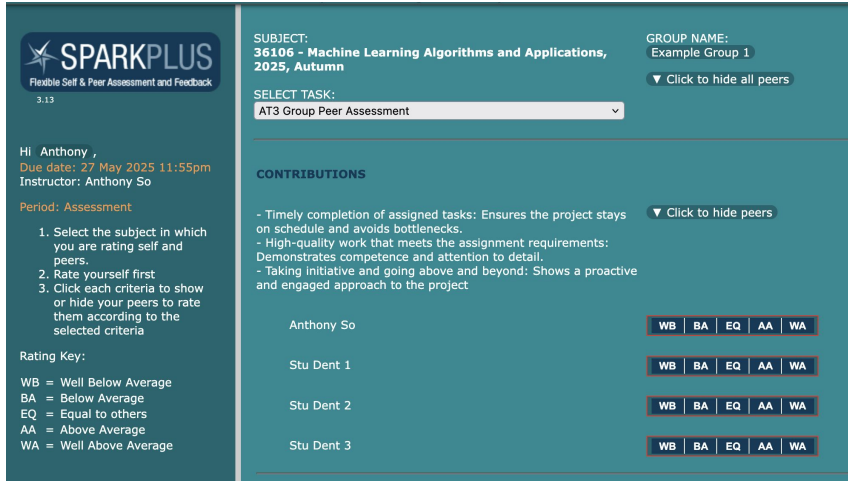
<https://spark.uts.edu.au>

Tool that combines self and peer assessment:

- ability to provide insights into collaborative and team based activities
- capture self and peer feedback support individual grades moderation in group work contribution

Open from 16 May 2025 to 27 May 2025 11:55pm

**It is mandatory for every student to fill the form.
Otherwise will incur penalties.**



The screenshot shows the SparkPlus assessment interface. On the left, there is a sidebar with the SparkPlus logo, a greeting 'Hi Anthony', the due date '27 May 2025 11:55pm', the instructor 'Anthony So', and a 'Period: Assessment' section with three steps: 1. Select the subject, 2. Rate yourself first, and 3. Click each criteria. Below this is a 'Rating Key' with five levels: WB (Well Below Average), BA (Below Average), EQ (Equal to others), AA (Above Average), and WA (Well Above Average). The main area displays the subject '36106 - Machine Learning Algorithms and Applications, 2025, Autumn' and the group name 'Example Group 1'. A 'SELECT TASK' dropdown menu is set to 'AT3 Group Peer Assessment'. Under the 'CONTRIBUTIONS' section, there are four criteria: 'Timely completion of assigned tasks', 'High-quality work that meets the assignment requirements', 'Demonstrates competence and attention to detail', and 'Taking initiative and going above and beyond'. Below these criteria, there are four rows of student names: 'Anthony So', 'Stu Dent 1', 'Stu Dent 2', and 'Stu Dent 3'. Each row has a set of five buttons labeled WB, BA, EQ, AA, and WA for rating.



Time for Lab

